DOCUMENT TECHNIQUE

Architecture de l'application

Fonctionnement low-level, librairies, interactions inter-modules et trajet d'une requête

ERP multi-institutions pour le transport de voyageurs (SCIG / Brainstore). Backend Django REST Framework, frontend Angular (standalone). Ce document décrit les couches, les dépendances techniques de chaque module et la circulation d'une requête de bout en bout.

Sommaire

1 Vue d'ensemble & pile technique
2 Architecture backend en couches
3 Sécurité, multi-tenant & droits (RBAC)
4 Architecture frontend (Angular)
5 Modules : fonctionnement low-level & librairies
6 Interactions entre modules
7 Trajet d'une requête (aller-retour)
8 Génération documentaire & annexes

1. Vue d'ensemble & pile technique

L'ERP est une application web multi-institutions (multi-tenant) pour une compagnie de transport de voyageurs. Elle se compose de deux briques : un backend d'API REST (Django REST Framework) et un frontend SPA (Angular standalone). La communication se fait exclusivement en JSON sur HTTP, sous le préfixe /api/v1/, avec authentification par jeton JWT.

Pile backend

`Django 5.1`	Framework web : ORM, migrations, middlewares, i18n, gestion des URLs.
`djangorestframework 3.15`	Couche API REST : APIView, serializers, permissions, pagination, exceptions.
`djangorestframework-simple jwt 5`	Authentification par jeton JWT (access/refresh) + claims (company_id, agency_id).
`psycopg 3`	Pilote PostgreSQL en production ; SQLite en développement.
`django-filter 24`	Filtres de liste déclaratifs (query params) sur les endpoints.
`drf-spectacular 0.27`	Génération du schéma OpenAPI et de la doc Swagger (/api/docs/).
`django-modeltranslation + django-parler`	Champs multilingues (FR/EN) et interface de traduction.
`django-environ`	Lecture de la configuration via variables d'environnement (.env).
`pyotp 2.9`	Codes 2FA (TOTP) pour la sécurité des comptes.
`segno 1.6`	Génération de QR codes (billets), 100% Python.
`weasyprint 63`	Rendu PDF (rapports, bons de dépense, reçus) à partir de HTML/CSS.
`openpyxl 3.1`	Export Excel (.xlsx) des rapports.
`Pillow`	Traitement d'images (logos d'institution).
`gunicorn + whitenoise`	Serveur WSGI de production + service des fichiers statiques.
`sentry-sdk`	Supervision des erreurs en production.
`boto3 + django-storages`	Stockage des médias sur S3 en production.

Pile frontend

`@angular/* 21 (standalone)`	Framework SPA : composants autonomes, router, forms, change detection.
`rxjs 7`	Programmation réactive : tous les appels HTTP renvoient des Observables.
`@angular/common/http`	HttpClient + intercepteurs fonctionnels (JWT, erreurs, désemballage).
`bootstrap 5 + @angular/cdk`	Grille, modales, composants UI de base.
`primeng + @primeng/themes`	Composants riches (tables, overlays).
`@angular/material + cdk`	Quelques composants et utilitaires (overlay, a11y).
`apexcharts / ng-apexcharts`	Graphiques des tableaux de bord.
`@fullcalendar/*`	Vues calendrier.
`leaflet`	Cartographie (suivi/points de service).
`ngx-mask, ngx-editor, @angular/cdk`	Saisies masquées, éditeur riche, upload, tags.
`date-fns`	Manipulation de dates.
`remixicon / tabler / fontawesome`	Jeux d'icônes.

2. Architecture backend en couches

Chaque module backend est une application Django autonome, organisée selon un patron de couches strict. Cela isole la logique métier des vues HTTP et des requêtes SQL, pour des composants testables un par un.

Couches d'un module (de l'extérieur vers la base)

- urls.py — déclare les routes du module (path -> vue).
- views.py — vues APIView : valident l'entrée (serializer), appliquent les permissions, orchestrent, et renvoient une réponse normalisée. Ne contiennent pas de logique métier lourde.
- serializers.py — validation et (dé)sérialisation des données entrantes/sortantes.
- services/ — logique métier d'écriture : créations/mises à jour, transactions atomiques, règles de gestion.
- selectors/ — logique de lecture : construction des requêtes (querysets), filtres, agrégations.
- models/ — modèles ORM (tables) ; héritent de BaseModel (id UUID, horodatage, soft-delete).

Conventions transverses (app core)

`core/responses.py`	success_response / created_response / error_response : enveloppe { success, data } ou { success, error }.
`core/exceptions.py`	custom_exception_handler : transforme toute exception DRF/ApplicationError en { success:false, error:{message,details} }.
`core/pagination.py`	StandardResultsSetPagination : listes paginées { success, data, pagination } (page_size=25).
`core/permissions.py`	IsAuthenticated, HasModulePermission (droits par méthode via permission_map).
`core/models.py`	BaseModel : clé UUID, created/updated, soft_delete(user=…).

Le routage global agrège les modules : config/urls.py monte api/v1/router.py, qui inclut les urls.py de chaque application sous un préfixe (ex. /api/v1/treasury/, /api/v1/voyages/trips/).

3. Sécurité, multi-tenant & droits (RBAC)

Authentification

L'authentification repose sur SimpleJWT. Le modèle utilisateur est authentication.User ; le backend EmailOrUsernameBackend autorise la connexion par email OU nom d'utilisateur. À la connexion, le jeton d'accès embarque des claims : identifiant utilisateur, company_id (institution), agency_id, et le statut administrateur d'institution.

Isolation multi-tenant

1. **TenantMiddleware**
   `apps/core/middleware.py`
   S'exécute avant l'authentification DRF : décode le JWT présent dans l'en-tête Authorization et pose l'institution courante (company_id) dans un ContextVar accessible partout via get_current_tenant().

2. **EntitlementMiddleware**
   `apps/core/entitlement_middleware.py`
   S'exécute après : si le chemin appartient à un module non inclus dans le package de l'institution, renvoie un 403 « Module non inclus dans votre offre » — même pour les vues qui redéfinissent leurs permissions.

Droits (RBAC)

Les vues déclarent permission_classes = [IsAuthenticated, HasModulePermission] et un permission_map par méthode HTTP (ex. {'GET': 'treasury.expense.view', 'POST': 'treasury.expense.manage'}). Les permissions sont regroupées en rôles (app admin_rbac, table de jointure RolePermission), eux-mêmes regroupés en profils.

Points d'attention RBAC

- Les administrateurs d'institution et les super-admins contournent la vérification des permissions.
- Une nouvelle permission n'est effective qu'après avoir été accordée au rôle ET reconnexion de l'utilisateur (les droits sont mis en cache côté client au login).
- Les rôles personnalisés sont créés via l'interface, pas pré-remplis par le code.

4. Architecture frontend (Angular)

Le frontend est une SPA Angular en composants standalone (sans NgModule). L'accès réseau est centralisé dans le dossier _api/ : configuration des URLs, services par domaine, intercepteurs et modèles TypeScript.

Organisation du dossier _api/ et des pages

- _api/config/api.config.ts — constantes d'URL (API_BASE + objets par module : TREASURY, VOYAGES…).
- _api/services/ — un service par domaine ; encapsule HttpClient et mappe la réponse (r.results ?? r.data ?? r).
- _api/interceptors/ — chaîne d'intercepteurs HTTP fonctionnels.
- pages/ — composants d'écran (listes, modales) qui consomment les services.
- shared/ — briques réutilisables (pagination, directives *hasPermission / *hasModule, autocomplete client…).

Chaîne d'intercepteurs HTTP

Déclarée dans app.config.ts : provideHttpClient(withInterceptors([jwtInterceptor, errorInterceptor, apiUnwrapInterceptor])). L'ordre compte : la requête traverse les intercepteurs de haut en bas, la réponse remonte de bas en haut.

`jwt.interceptor.ts`	Ajoute l'en-tête Authorization: Bearer <access> aux appels /api (sauf URLs publiques).
`error.interceptor.ts`	Sur 401, rafraîchit le jeton (refresh mutualisé, single-flight) et rejoue la requête ; redirige vers la connexion si le refresh échoue.
`api-unwrap.interceptor.ts`	Désemballe { success, data } en donnée brute et normalise les erreurs en { detail }.

La sécurité d'affichage est doublée côté client : les gardes de route protègent les pages, et les directives structurelles *hasPermission / *hasModule masquent boutons et menus selon les droits — la vérification faisant autorité restant toujours celle du backend.

5. Modules : fonctionnement low-level & librairies

Tous les modules suivent le patron urls -> views -> services/selectors -> models. Cette section précise le

rôle, les données clés et les dépendances spécifiques de chacun.

core

Socle commun : BaseModel (UUID, soft-delete), middlewares multi-tenant, helpers de réponse, permissions, pagination, gestionnaire d'exceptions. Toutes les autres apps en dépendent.

authentication

Modèle User, connexion (SimpleJWT), backend email-ou-username, 2FA (pyotp). Émet les jetons portant les claims d'institution/agence utilisés par le TenantMiddleware.

system_config

Institutions (Company), Agences, Packages/Abonnements, Paramètres clé-valeur (Parameter) et Feature Flags. Le service create_company initialise une institution : rôles par défaut, plan comptable OHADA, correspondances de trésorerie, moyens de paiement, types d'équipage. Source du multi-tenant.

admin_rbac

Rôles, Permissions, Profils et la jointure RolePermission. Les permissions sont semées par des migrations de données. HasModulePermission (dans core) s'appuie sur ces tables.

fleet

Parc de véhicules (marques, modèles, catégories), maintenance (services, contrats, documents, accidents), exploitation (missions/Assignment, carburant), et référentiels techniques. Fournit l'éligibilité des véhicules aux voyages (disponibilité, plan de bus actif, absence de conflit d'intervention ou de mission).

voyages_config

Référentiel du réseau : villes, points de service, trajets (Route), plages horaires, planifications (ServicePattern), plans de bus (SeatingPlan) et affectations, types de colis, paramètres. Alimente la création et la tarification des voyages.

voyages_trips

Cœur de l'exploitation : Trip (cycle Brouillon->Soumis->Parti->Terminé), sièges (TripSeat), étapes, équipage (TripCrew), barème de dépenses, et transbordements. Sélecteur d'éligibilité qui interroge la flotte ; création depuis une planification (horaires dérivés).

voyages_pricing

Catégories tarifaires et tarifs (origine/destination/catégorie/type -> montant). resolve_fare est utilisé par les réservations pour calculer le prix.

voyages_booking

Passagers (client à champ Nom unique), réservations (Booking) et billets (Ticket), aller-retours, fidélité. À l'encaissement, appelle la trésorerie (record_booking_payment). L'annulation libère les sièges, annule les bagages liés et déclenche un remboursement selon le taux configuré.

voyages_consignment

Colis (Parcel) et bagages (Baggage) : enregistrement, affectation à un voyage, manifeste, réception/retrait, annulation. À l'enregistrement payé, appelle la trésorerie (record_parcel_payment / record_baggage_payment) ; génération de QR/étiquettes via segno.

treasury

Caisses (CashRegister) et sessions, comptes bancaires, transactions (Transaction : income/expense par catégorie), transferts, dépenses (voyage/générale), recettes, moyens de paiement configurables et taux de remboursement. create_transaction met à jour les soldes ET poste l'écriture comptable via le pont comptable.

accounting

Plan comptable OHADA (Account), journaux, exercices/périodes, écritures (JournalEntry). Le pont TreasuryAccountingBridge convertit chaque mouvement de trésorerie en écriture équilibrée via une correspondance catégorie -> compte (MovementAccountMapping).

reporting

Registre de rapports : chaque rapport est une fonction build(company, filtres) enregistrée via un décorateur, renvoyant une structure ReportResult rendue en PDF (weasyprint), CSV ou Excel (openpyxl). Lit les données de la trésorerie, des dépenses et des voyages.

online_booking

Canal public de réservation (recherche + réservation côté client), s'appuyant sur les mêmes services de réservation et de trésorerie que le guichet.

sync_scig_sage

Interface avec Sage 100 (dépôts, inventaires). Dépendances techniques serveur : pilote SQL (pyodbc) et chiffrement des identifiants — à provisionner en production si la synchro est utilisée.

hr

Ressources humaines (employés). Application présente côté backend mais non exposée dans la barre latérale (voir la rétrospection).

6. Interactions entre modules

Les modules ne s'appellent pas au hasard : les dépendances suivent des arêtes précises, presque toujours orientées vers la trésorerie et la comptabilité pour la valeur, et vers system_config/core pour le socle.

Principales arêtes de dépendance

- system_config -> tous : create_company initialise les référentiels de chaque module.
- core (middlewares) -> tous : isolation par institution et blocage des modules hors offre, à chaque requête.
- admin_rbac -> tous (via core) : chaque vue vérifie une permission.
- voyages_trips -> fleet : éligibilité du véhicule (disponibilité, plan actif, conflits de mission/intervention).
- voyages_trips -> voyages_config : trajets, plans de bus, planifications, étapes.
- voyages_booking -> voyages_pricing : calcul du tarif (resolve_fare).
- voyages_booking -> treasury : enregistrement de la recette à l'encaissement (record_booking_payment).
- voyages_consignment -> treasury : recette colis/bagage (record_parcel_payment / record_baggage_payment).
- voyages_booking / voyages_consignment -> treasury : remboursement à l'annulation (refund_on_cancel).
- treasury -> accounting : chaque transaction poste une écriture (TreasuryAccountingBridge).
- reporting -> treasury / voyages_trips : lecture agrégée pour les états.
- online_booking -> voyages_booking + treasury : réutilise les mêmes services que le guichet.

Règles de couplage

- Le sens de la valeur est unidirectionnel : ventes et dépenses « descendent » vers la trésorerie, qui « descend » vers la comptabilité. La comptabilité ne rappelle jamais la trésorerie.
- Les appels inter-modules passent par les services (jamais directement par les modèles d'une autre app), ce qui garde les règles métier au même endroit.

7. Trajet d'une requête (aller-retour)

Exemple concret et représentatif : l'agent ouvre « Dépenses générales » et la liste se charge. On suit la requête depuis le clic jusqu'à l'affichage, couche par couche et fichier par fichier.

Aller — du composant au backend

1 — Le composant déclenche l'appel

`angular/src/_api/pages/treasury/expenses/general/general-expenses.component.ts`

ngOnInit/reload() appelle le service : this.svc.list({ status, date_from, date_to }).

2 — Le service construit la requête HTTP

`angular/src/_api/services/treasury/general-expense.service.ts`

HttpClient.get(TREASURY.expGeneral, { params }). L'URL vient de la config centralisée.

3 — La configuration fournit l'URL

`angular/src/_api/config/api.config.ts`

TREASURY.expGeneral = `${API_BASE}/treasury/expenses/general/`.

4 **Intercepteur JWT**

`angular/src/_api/interceptors/jwt.interceptor.ts`

Ajoute Authorization: Bearer <access> (jeton lu depuis TokenService).

5 **Intercepteur d'erreurs**

`angular/src/_api/interceptors/error.interceptor.ts`

Se met en veille sur la réponse ; interviendra seulement en cas de 401 (refresh).

6 **Sortie réseau**

`provideHttpClient(withInterceptors([...])) — app.config.ts`

La requête part vers le backend Django.

Traitement — côté Django

1 **Middlewares d'infrastructure**

`config/settings/base.py (MIDDLEWARE)`

Security, CORS, Session, Locale, Common, CSRF, Auth, Messages, XFrame s'appliquent.

2 **TenantMiddleware**

`apps/core/middleware.py`

Décode le JWT, extrait company_id et le pose dans le contexte (get_current_tenant).

3 **EntitlementMiddleware**

`apps/core/entitlement_middleware.py`

Vérifie que le module « treasury » est inclus dans l'offre de l'institution, sinon 403.

4 **Résolution d'URL**

`config/urls.py -> api/v1/router.py -> apps/treasury/urls.py`

Le chemin /api/v1/treasury/expenses/general/ est routé vers GeneralExpenseListView.

5 **Authentification & permissions DRF**

`core/permissions.py (IsAuthenticated, HasModulePermission)`

JWTAuthentication renseigne request.user ; la permission 'treasury.expense.view' est vérifiée (permission_map).

6 **La vue orchestre**

`apps/treasury/views.py -> GeneralExpenseListView.get`

Récupère l'institution, applique le périmètre d'agence, délègue la lecture au sélecteur.

7 **Le sélecteur lit les données**

`apps/treasury/selectors/expenses.py (list_general_expenses, list_refunds)`

Construit les querysets (dépenses + remboursements), filtre par date/statut/agence.

8 **Modèles & base de données**

`apps/treasury/models/ (GeneralExpense, Transaction)`

L'ORM traduit en SQL exécuté par psycopg sur PostgreSQL.

9 **Sérialisation & enveloppe**

`serializers.py + core/responses.py + core/pagination.py`

Les lignes sont sérialisées puis paginées : { success:true, data:[...], pagination:{...} }.

Retour — du backend à l'écran

1 **Réponse HTTP**

`StandardResultsSetPagination.get_paginated_response`

Le corps { success, data, pagination } (ou { success:false, error } via custom_exception_handler) repart.

2 **Intercepteur d'erreurs (retour)**

`error.interceptor.ts`

Si 401 : rafraîchit le jeton et rejoue la requête ; sinon laisse passer.

3 **Intercepteur de désemballage**

`api-unwrap.interceptor.ts`

Détecte { success, data } et renvoie au service la donnée brute (data) ; normalise toute erreur en { detail }.

4 **Le service adapte**

`general-expense.service.ts`

map(r => r.results ?? r.data ?? r) : renvoie un tableau simple au composant.

5 **Le composant met à jour l'état**

`general-expenses.component.ts`

this.rows = data ; la détection de changement d'Angular redessine le tableau (pagination, filtres).

Variantes

- Écriture (POST/PATCH) : même trajet, mais la vue délègue à un service (services/…) qui exécute la règle métier dans une transaction atomique (create_transaction, etc.) au lieu d'un sélecteur en lecture.
- Toute exception est convertie en { success:false, error:{ message, details } } par custom_exception_handler, puis en { detail } côté client — les composants affichent un message uniforme.

8. Génération documentaire & annexes

Les documents imprimables (rapports, bons de dépense, reçus de recette) sont produits côté serveur par WeasyPrint (HTML/CSS -> PDF) ; les exports tableurs par openpyxl. WeasyPrint requiert des librairies système (Pango/Cairo) installées dans l'image de production ; en leur absence, l'endpoint renvoie un message explicite plutôt qu'une erreur opaque.

Chaînes de production de documents

- PDF documents : weasyprint (letterhead commun + gabarits par document).
- PDF rapports : registre reporting -> render/pdf.py.
- Excel : openpyxl (render/xlsx.py) ; CSV : module standard (render/csv_.py).
- QR codes billets/colis : segno.
- Documentation d'API : drf-spectacular (Swagger sous /api/docs/).

Déploiement : gunicorn sert l'application, whitenoise les fichiers statiques, les médias vont sur S3 (django-storages/boto3), les migrations sont appliquées au démarrage, et sentry-sdk supervise les erreurs.